Smart Desk Lamp Text Reader

Instruction Manual | ESP32-S3 camera, OCR and Google search | Version 1.0, October 2026

1. What this project is

This project turns an ordinary desk lamp into a text reader. An ESP32-S3 camera board looks down at your desk. Put a book, a label or a printed page under the lamp, press SPACE on your laptop, and the system takes a photo, reads the text in it (OCR), lets you correct the text, and can search it on Google.

The lamp needs only two cables: one USB-C cable between the lamp's ESP32 board and your laptop (it powers the board, the camera and the screen), and the lamp's own power cable for its light. The two are completely separate.

How it works

1. The ESP32 camera sends a live picture to your laptop over the USB cable.
2. A Python program on the laptop shows the live video in a window.
3. You press SPACE or click the video. The ESP32 takes a larger, sharper photo (1280 x 960).
4. EasyOCR, running on your laptop, reads the text in the photo.
5. The terminal shows the text and asks whether you want to correct it and whether to search it on Google.
6. The small TFT screen on the lamp is meant to show the live view, the photo and the text too (see Section 8).

Status at the time of writing. Working and tested: camera, USB link, live video, photo capture, OCR (a book cover was read correctly), text correction, Google search, and saving photos. Not working yet: the 2.8-inch TFT screen only shows its white backlight. Every other function works without it. Section 8 explains what has been tried and what to try next.

2. Parts and software

Hardware

Part	Details
ESP32-S3 camera board	ESP32-S3-WROOM-1 module, N16R8 (16 MB flash, 8 MB PSRAM), flat-cable camera connector, two USB-C ports. Our camera sensor was detected as a GC2145.
TFT screen	2.8-inch SPI display, 240 x 320, marked KMRTM28028-SPI. It also has touch pins and a microSD slot, which this project does not use.
Breadboard and jumper wires	Male-to-female and male-to-male jumpers.
USB-C data cable	Must be a data cable, not a charge-only one.
Laptop	Windows laptop (tested). macOS and Linux also work; only the port name changes.
Desk lamp	Holds the board so the camera looks down at the desk.
microSD card (optional)	Goes in the board's own SD slot. Photos are also copied there.

Software

Software	Used for
Arduino IDE 2.x	Writing the program onto the ESP32 board
ESP32 board package (Espressif)	Lets Arduino IDE understand the ESP32-S3
LovyanGFX library	Drives the TFT screen
Python 3.10, 3.11 or 3.12	Runs the laptop program
Visual Studio Code	Editor and terminal for Python
pyserial, easyocr, opencv-python, numpy	Python libraries the program needs

Files

- `esp32_camera_tft.ino` is the program for the ESP32 board.
- `esp_ocr_terminal.py` is the program for the laptop.

- `tft_test.ino` and `tft_raw_test.ino` are small screen tests.

3. Circuit connection manual

Always unplug the USB cable before you change any wire.

3.1 Screen to board

The screen has a row of labelled pins. Connect them as follows.

Screen pin	Connect to	Notes
VCC	3V3	Power for the screen
LED	3V3	Backlight; share the same 3V3 strip as VCC
GND	GND	Any GND pin on the board
CS	GND	Tied low so the screen is always selected. Alternative: a real wire to GPIO 1
RESET	3V3	Tied high. Alternative: a real wire to GPIO 2
D/C	GPIO 42	Data/command
SDI (MOSI)	GPIO 21	Data into the screen
SCK	GPIO 47	Clock
SDO (MISO)	not connected	Not needed
T_CLK, T_CS, T_DIN, T_OUT, T_IRQ	not connected	Touch is not used

If you use the alternatives for CS and RESET, change `PIN_TFT_CS` to 1 and `PIN_TFT_RST` to 2 near the top of the sketch.

3.2 Breadboard rules

- The board has only one 3V3 pin. Run one jumper from it to a breadboard strip, then connect VCC, LED and RESET to that same strip with short jumpers. Do the same for GND and CS.

- Each row of the breadboard has two halves, A to E and F to J, and they are not connected to each other. Both wires of a connection must be on the same half of the same row.
- Push each jumper in firmly. A loose wire looks identical to a good one and is the most common fault.

3.3 Pins you must not use

The camera, SD card and memory use many pins on this board. Do not wire the screen to:

- GPIO 4 to 13 and 15 to 18 (camera)
- GPIO 35 to 40 (memory chip and SD card)
- GPIO 19 and 20 (USB)
- GPIO 0, 3, 45 and 46 (start-up pins)

Free pins that suit the screen: 1, 2, 14, 21, 41, 42, 47 and 48. Note that GPIO 2 also drives a small LED on the board.

3.4 Camera

The camera module plugs into the board's flat-cable (FPC) connector. Make sure the cable is fully inserted and the small latch is pressed closed. If the program later reports CAMERA ERROR, check this first.

4. Installing the Arduino IDE and uploading the sketch

1. Download Arduino IDE 2.x from arduino.cc/en/software and install it with the default options.
2. Open Arduino IDE. Open Boards Manager (the board icon in the left bar), search for esp32, and install **esp32 by Espressif Systems**.
3. Open Library Manager (the books icon), search for LovyanGFX, and install the one by lovyan03.
4. Create a folder named `esp32_camera_tft`, put `esp32_camera_tft.ino` inside it, and open the file in Arduino IDE. (The folder name must match the file name.)
5. Open the Tools menu and set:
   - Board: `ESP32S3 Dev Module`
   - USB CDC On Boot: `Enabled`
   - PSRAM: `OPI PSRAM`

- Flash Size: `16MB` (this board is N16)
- Partition Scheme: the default, or `Huge APP`
- Core Debug Level: `None`

6. Plug the USB-C cable into the board and choose its port under Tools, then Port.
7. Click the Upload arrow. Wait for Done uploading.
8. If the upload will not start, hold the BOOT button, tap RST, release BOOT, and try again.

Near the top of the sketch is the pin list (`PIN_TFT_...`). It must match your wires exactly.

The Serial Monitor (115200 baud) should show a line such as `CAMERA_OK MODE=RGB565 PID=0x2145` and then `READY SD=1` (or `SD=0` with no card). The line `CAM_ERR 0x106 in JPEG mode` is normal for this camera sensor: it cannot make JPEG pictures itself, so the sketch switches to a mode where the ESP32 converts the picture.

Close the Serial Monitor before running the Python program. Only one program can use the port at a time.

5. Installing Python and Visual Studio Code

1. Download Python from python.org/downloads (3.10, 3.11 or 3.12). In the installer tick **Add python.exe to PATH**, then install.
2. Download Visual Studio Code from code.visualstudio.com and install it.
3. In VS Code open Extensions (Ctrl+Shift+X), search for Python, and install the one from Microsoft.
4. Create a folder for the project, put `esp_ocr_terminal.py` in it, and open that folder with File, then Open Folder.
5. Open a terminal with Terminal, then New Terminal.
6. Install the libraries:

```
pip install pyserial easyocr opencv-python numpy
```

This is a large download because EasyOCR installs PyTorch. The first time the program runs it also downloads the text-reading model. After that, reading works without internet.

7. If the video window will not open and the program mentions the headless version of OpenCV, run:

```
pip uninstall -y opencv-python opencv-python-headless
pip install opencv-python
```

6. Running the project

1. Wire the circuit, plug the USB-C cable into the lamp board, and make sure the Serial Monitor is closed.
2. Find the port: it is shown in Arduino IDE under Tools, then Port, or in Windows Device Manager under Ports (COM and LPT). In our build it was COM6.
3. In the VS Code terminal run:

```
python esp_ocr_terminal.py --port COM6
```

4. The program opens its own terminal window and loads EasyOCR, which takes a moment. Then a window called ESP32 Live Camera shows the live picture.
5. Place a printed page under the lamp, 15 to 25 cm from the camera. Click the video window and press SPACE (or click the video).
6. The photo takes a few seconds, because the camera switches to full size. Then the text appears in the terminal window.
7. Click the terminal window. It asks **Do you want to edit/correct this text? (y/n)**. Answer y to type a corrected version, or n to keep the text.
8. It asks **Search this on Google? (y/n)**. Pressing ENTER counts as yes, and your browser opens the search.
9. Back in the video window, press SPACE for the next photo, or Q or ESC to quit.

Photos are saved in a folder called `captures` next to the Python file, with the date and time in the file name. If a microSD card is in the board, a copy is also saved there as `img_00001.jpg`, `img_00002.jpg` and so on.

To read other languages, change `LANGUAGES = ["en"]` at the top of the Python file, for example to `["en", "hi"]` for English and Hindi.

7. Tips for clear reads

- Hold the text 15 to 25 cm from the camera. The lens is fixed focus; if it looks blurry you can turn the lens barrel slightly.
- Use bright, even light. Avoid glare on glossy paper by tilting the page.
- Keep the page flat and fill a good part of the picture with text.
- Printed text works best. Decorative cover fonts and handwriting are less reliable. In our test a book cover was read correctly except for the decorative artwork at the end.

- If the picture is upside down or mirrored, remove the // from the `set_vflip` or `set_hmirror` lines in the sketch (in `applySensorSettings`) and upload again.
- Keep the lens clean.

8. The TFT screen: status and next steps

What the screen is for. Once working, the screen shows the live camera view, the captured photo, and text pages: the text found, the corrected text, and status messages.

Current status. The backlight turns on, but the screen only shows white. We ran three programs: the full sketch, a screen test using LovyanGFX, and a raw test that uses no library at all. All three ran without errors, but the screen stayed white. That means the program is fine and the screen is not receiving or accepting the commands. This points to hardware, but we have not yet proved which part.

What to try next, one change at a time:

1. Power the screen from the board's 5V pin instead of 3V3. Keep LED and RESET on 3V3.
2. Replace the tied CS and RESET with real wires to GPIO 1 and GPIO 2, and set the two pin numbers in the sketch.
3. Test every screen wire for continuity with a multimeter, with USB unplugged.
4. In `tft_test.ino` set `PANEL_TYPE` to 2, and in `tft_raw_test.ino` set `CONTROLLER` to 2, in case the screen uses a different controller chip.
5. Check the module's J1 solder pads near the regulator. Bridging them runs the module directly from 3.3 V. Read the module documentation before soldering.
6. If nothing works, the module may be faulty. Try a second screen.

Do not worry if the screen stays white: everything else in this manual works without it.

9. Troubleshooting

Symptom	Likely cause	What to do
Could not open COM6, or the board did not answer	Serial Monitor still open, wrong port, or a charge-only cable	Close the Serial Monitor, check the port number, use a data cable, try the other USB-C port

Symptom	Likely cause	What to do
The board did not send a live frame	Wrong sketch uploaded, or the camera did not start	Upload `esp32_camera_tft.ino`, set Core Debug Level to None, and read the Serial Monitor for CAM_ERR
CAMERA ERROR	Camera cable loose, or wrong PSRAM setting	Reseat the flat cable and close the latch; try QSPI PSRAM instead of OPI PSRAM
CAM_ERR 0x106 in JPEG mode	Normal for this camera sensor	Nothing; the sketch switches mode by itself
Photo was damaged in transfer	An older sketch cut photos at 128 KB	Upload the current sketch
No text detected	Blur, glare, poor light, text too small, or an upside-down picture	See Section 7
Video window does not open	The headless version of OpenCV is installed	Section 5, step 7
First start is very slow	EasyOCR is loading or downloading its model	Wait; later starts are faster
Board restarts at random	Weak USB cable, hub or port	Use a good data cable plugged straight into the laptop
A photo takes several seconds	The camera restarts at full size and converts the picture	Normal
The screen shows ? instead of a letter	The screen shows plain English letters only	Normal; the full text appears in the terminal

10. Questions and answers

What do I need to switch it on? One USB-C cable from the lamp's board to the laptop, plus the lamp's own power cable for its light. The board, camera and screen take their power from the USB cable. The lamp's light and the electronics do not depend on each other.

Is it safe? The electronics run on 5 V from USB, which is safe to touch. The lamp's light uses mains power. Keep the breadboard and wires away from the mains cable, the bulb holder and any exposed metal. Never connect the electronics to the mains, and have a qualified person handle any change to the lamp's mains wiring.

Does it work without a laptop? Not in this version. The ESP32 only takes pictures; the laptop reads the text. A future version could use a small computer such as a Raspberry Pi instead of the laptop.

Does it need the internet? Only for the first installation (downloads) and to open Google. After the reading model has been downloaded, OCR works offline.

Which languages can it read? EasyOCR supports many languages. Change the `LANGUAGES` line in the Python file, for example to English and Hindi. The small TFT screen can only show plain English letters.

Can it read handwriting? Only poorly. It works best on printed text.

Where are my photos stored? On the laptop in the `captures` folder, and on the microSD card in the board if one is fitted.

Is my data private? The photos and the text reading stay on your own laptop and SD card. Text is sent to Google only when you answer y to the search question.

Can I use a Mac or Linux computer? Yes. Install the same Python libraries. The port is named like /dev/ttyACM0 on Linux and /dev/cu.usbmodem followed by numbers on macOS. On Linux you may need to add your user to the dialout group.

Why does a photo take several seconds? This camera sensor cannot change picture size while running. For each photo it restarts at full size, takes the picture, converts it to JPEG, and then returns to live view.

Can I get a sharper photo? The sketch uses 1280 x 960. The sensor can go higher, but that uses more memory and time. The setting is `photoSize` in the sketch; change it only if you are comfortable testing.

Does the touchscreen work? No. The touch wires are not connected and the program does not use touch.

Can I use another ESP32 camera board? Possibly. The sketch has pin lists for the Freenove-style board, the XIAO ESP32S3 Sense and a custom board. Check your board's pin table. The XIAO has few free pins for a screen.

The text has small mistakes. What now? Answer y to the correction question and type the right text before searching.

Can it read QR codes or barcodes? With some extra Python code. See Section 11.

11. Advantages and what else you can build

Advantages

- **One cable.** USB gives the camera power and carries the pictures.
- **Low cost.** One small board, one screen, and a lamp you already own.
- **A fixed overhead camera.** Like a document camera, pages are always at a similar distance and angle.
- **The laptop does the hard work.** Reading text, or any other image task, runs on the laptop, which is far more powerful than the ESP32.
- **Anything Python can do with a picture can be added.** Any code, algorithm or AI model can sit behind the camera.
- **Open and hackable.** Both programs are plain text files that you can change.
- **Offline OCR and saved photos.** Every photo is saved with the date and time.
- **One piece of hardware, many projects.** Only the Python program changes.

The idea. The lamp is a USB camera that your laptop can use at any time. The Python program receives a photo and whatever you add after that is up to you. The hardware and the ESP32 program stay the same.

Example projects. Each of these is an idea that needs extra Python code. None of them is included in this build.

1. Rubik's Cube solver. Hold each of the six faces of a cube under the lamp and press SPACE to photograph it. A program then:

1. crops each photo to the 3 x 3 grid of stickers,
2. measures the average colour of each of the nine squares and classifies it as white, red, blue, orange, green or yellow (HSV colour values work well),
3. joins the six faces into one 54-letter description of the cube, in the order the solver expects, and
4. passes it to the `kociemba` library (`pip install kociemba`), which returns the sequence of moves.

The program prints the moves, and the lamp's screen could show them one at a time. Even lighting and a quick colour calibration matter a lot; orange and red are the hardest colours to tell apart.

```
import kociemba
cube = "...54 letters, nine per face, in the order U R F D L B..."
print(kociemba.solve(cube))
```

2. Read-aloud for people with low vision. Send the OCR text to a text-to-speech library such as pyttsx3.

3. Instant translator. Pass the OCR text to a translation library and show the result in the terminal or on the screen.

4. Searchable notes and document archive. Save each page as a text file, then build a searchable folder or a PDF.

5. Math homework helper. Read a printed equation and solve it with SymPy.

6. Sudoku solver. Photograph a puzzle, read the digits, and solve it with a standard algorithm.

7. QR code and barcode scanner. Read codes with OpenCV or pyzbar, then look up the product or open the link.

8. Label and inventory logger. Read part numbers or serial numbers and add each one to a spreadsheet file with the time.

9. Business card reader. Pull names, phone numbers and emails into a contacts file.

10. Counter and sorter. Count coins or objects, or sort items by colour, with OpenCV.

11. Time-lapse recorder. Photograph a desk project or a plant every few minutes.

12. Flashcard maker. Photograph a textbook page and turn key lines into study cards.

To add any of these, replace the text-reading step inside the `capture_and_read` function of the Python file with your own code.

12. Care and safety

- Unplug the USB cable before changing wires. Do not move jumpers while the board is powered.
- Keep liquids away from the board and screen.
- Keep low-voltage wires well apart from the lamp's mains wiring.
- The board and screen may get slightly warm. Do not cover them.

- Touch a metal object before handling the board, to discharge static. Do not touch the camera lens; clean it with a soft microfibre cloth.
- Do not bend the camera's flat cable sharply, and keep its latch closed.
- For the optional microSD card, use a FAT32-formatted card (32 GB or smaller is the safest choice).

13. Quick reference

Start the program

```
python esp_ocr_terminal.py --port COM6
```

Keys in the video window

Key	Action
SPACE, ENTER or mouse click	Take a photo and read the text
Q or ESC	Quit

Commands the Python program sends to the board

Letter	Meaning
P	Ping; the board answers PONG with SD=1 or SD=0
V	Send one live preview frame (320 x 240)
C	Take a full photo, save it to the SD card, and send it
T	Show a page of text on the TFT screen

Message format (for developers)

- Photo reply: the letters IMG!, then 4 bytes giving the length (little-endian), then the JPEG bytes.
- Error reply: the letters ERR!
- Text command: the letter T, then 2 bytes giving the length (little-endian), then ASCII text. The first line is the title and the rest is the body.

Pin list used by this build

SCK = GPIO 47, SDI (MOSI) = GPIO 21, D/C = GPIO 42, CS = GND (or GPIO 1), RESET = 3V3 (or GPIO 2), VCC and LED = 3V3, GND = GND.

End of manual.